Part 5 — Kubernetes Deep Dive

Probability: 🔴 90% — Very Frequently Asked

Interview-Ready YAMLs

Pod (Foundation of Everything)

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  namespace: default
  labels:                           # Labels = how K8s identifies pods
    app: nginx                      # Used by Services, Deployments
    version: "1.0"
spec:
  containers:
  - name: nginx
    image: nginx:1.25-alpine        # Always pin versions, never :latest
    ports:
    - containerPort: 80
    resources:                      # ALWAYS set in production
      requests:                     # Scheduling guarantee
        memory: "64Mi"
        cpu: "100m"                 # 100 millicores = 0.1 CPU
      limits:                       # Hard ceiling
        memory: "128Mi"
        cpu: "250m"
    livenessProbe:                  # Is the container alive?
      httpGet:
        path: /healthz
        port: 80
      initialDelaySeconds: 10
      periodSeconds: 10
    readinessProbe:                 # Is it ready to serve traffic?
      httpGet:
        path: /ready
        port: 80
      initialDelaySeconds: 5
      periodSeconds: 5
  restartPolicy: Always             # Always | OnFailure | Never
```

Q: "Should you create Pods directly?"

> *"Never in production. Always use Deployments (or StatefulSets/DaemonSets). Bare pods aren't rescheduled if a node fails. Deployments provide self-healing, rolling updates, and scaling."*

Deployment + ReplicaSet

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
  labels:
    app: web-app
spec:
```

```
  replicas: 3                       # Desired pod count
  selector:
    matchLabels:
      app: web-app                  # MUST match template labels
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0             # Zero-downtime: keep all old pods until new ones ready
      maxSurge: 1                   # Create 1 extra pod at a time
  template:                         # Pod template
    metadata:
      labels:
        app: web-app
    spec:
      containers:
      - name: web-app
        image: myregistry/web-app:v2.1.0
        ports:
        - containerPort: 8080
        env:
        - name: DB_HOST
          valueFrom:
            configMapKeyRef:
              name: app-config
              key: db_host
        - name: DB_PASSWORD
          valueFrom:
            secretKeyRef:
              name: app-secrets
              key: db_password
        resources:
          requests: { memory: "128Mi", cpu: "100m" }
          limits:   { memory: "256Mi", cpu: "500m" }
        readinessProbe:
          httpGet: { path: /health, port: 8080 }
          initialDelaySeconds: 5
          periodSeconds: 10
        livenessProbe:
          httpGet: { path: /health, port: 8080 }
          initialDelaySeconds: 15
          periodSeconds: 20
```

Q: "What's the relationship between Deployment, ReplicaSet, and Pod?"

```
Deployment (manages)
  └── ReplicaSet (ensures N replicas running)
       ├── Pod 1
       ├── Pod 2
       └── Pod 3
```

> *"Deployment manages ReplicaSets. When you update image, Deployment creates NEW ReplicaSet, scales it up while scaling OLD one down. This enables rollback — old ReplicaSet is kept (scaled to 0). You never manage ReplicaSets directly."*

Services (ClusterIP, NodePort, LoadBalancer)

```
# ClusterIP — Internal only (default)
apiVersion: v1
kind: Service
metadata:
  name: web-app-svc
spec:
```

```
  type: ClusterIP                    # Only accessible within cluster
  selector:
    app: web-app                     # Matches pod labels
  ports:
  - port: 80                        # Service port
    targetPort: 8080                 # Container port
---
# NodePort — Expose on node IP
apiVersion: v1
kind: Service
metadata:
  name: web-app-nodeport
spec:
  type: NodePort
  selector:
    app: web-app
  ports:
  - port: 80
    targetPort: 8080
    nodePort: 30080                  # External port (30000-32767)
---
# LoadBalancer — Cloud load balancer
apiVersion: v1
kind: Service
metadata:
  name: web-app-lb
spec:
  type: LoadBalancer                 # Creates cloud LB (ALB/NLB/Azure LB)
  selector:
    app: web-app
  ports:
  - port: 80
    targetPort: 8080
```

Q: "ClusterIP vs NodePort vs LoadBalancer — when to use each?"

Type	Access	Use Case
ClusterIP	Internal only	Service-to-service communication
NodePort	`<NodeIP>:<30000-32767>`	Dev/testing, bare-metal
LoadBalancer	External cloud LB	Production external access
Ingress	HTTP/HTTPS routing	Production — one LB, multiple services

Ingress with TLS

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: web-ingress
  annotations:
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
    cert-manager.io/cluster-issuer: "letsencrypt-prod"
spec:
  ingressClassName: nginx
  tls:
  - hosts:
    - app.example.com
    secretName: app-tls-cert         # TLS cert stored as K8s Secret
  rules:
  - host: app.example.com
    http:
      paths:
      - path: /
```

```
        pathType: Prefix
        backend:
          service:
            name: web-app-svc
            port:
              number: 80
      - path: /api
        pathType: Prefix
        backend:
          service:
            name: api-svc
            port:
              number: 8080
```

StatefulSet (for databases)

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
spec:
  serviceName: "postgres"            # Headless service for stable DNS
  replicas: 3
  selector:
    matchLabels: { app: postgres }
  template:
    metadata:
      labels: { app: postgres }
    spec:
      containers:
      - name: postgres
        image: postgres:15-alpine
        ports: [{ containerPort: 5432 }]
        volumeMounts:
        - name: pgdata
          mountPath: /var/lib/postgresql/data
  volumeClaimTemplates:              # Each pod gets its OWN PVC
  - metadata:
      name: pgdata
    spec:
      accessModes: ["ReadWriteOnce"]
      storageClassName: managed-premium
      resources:
        requests: { storage: 50Gi }
```

Q: "Deployment vs StatefulSet?"

Feature	Deployment	StatefulSet
Pod identity	Random names	Ordered: `postgres-0`, `postgres-1`
Storage	Shared or none	Each pod gets unique PVC
Startup order	Parallel	Sequential (0 → 1 → 2)
Stable DNS	No	Yes: `postgres-0.postgres.namespace.svc`
Use case	Stateless apps	Databases, Kafka, ZooKeeper

DaemonSet, Job, CronJob

```
# DaemonSet — One pod per node (monitoring agents, log collectors)
apiVersion: apps/v1
kind: DaemonSet
metadata:
```

```
  name: fluent-bit
spec:
  selector:
    matchLabels: { app: fluent-bit }
  template:
    metadata:
      labels: { app: fluent-bit }
    spec:
      containers:
      - name: fluent-bit
        image: fluent/fluent-bit:2.1
        volumeMounts:
        - name: varlog
          mountPath: /var/log
      volumes:
      - name: varlog
        hostPath: { path: /var/log }
---
# Job — Run to completion
apiVersion: batch/v1
kind: Job
metadata:
  name: db-migration
spec:
  backoffLimit: 3                    # Retry 3 times on failure
  template:
    spec:
      containers:
      - name: migrate
        image: myapp/migrate:v1
        command: ["python", "manage.py", "migrate"]
      restartPolicy: Never
---
# CronJob — Scheduled job
apiVersion: batch/v1
kind: CronJob
metadata:
  name: backup
spec:
  schedule: "0 2 * * *"             # Daily at 2 AM
  concurrencyPolicy: Forbid          # Don't overlap runs
  jobTemplate:
    spec:
      template:
        spec:
          containers:
          - name: backup
            image: myapp/backup:v1
          restartPolicy: OnFailure
```

ConfigMap & Secret

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  db_host: "postgres-svc"
  log_level: "info"
  config.yaml: |                     # File-based config
    server:
      port: 8080
      timeout: 30s
---
apiVersion: v1
kind: Secret
```

```
metadata:
  name: app-secrets
type: Opaque
data:
  db_password: cGFzc3dvcmQxMjM=     # base64 encoded (NOT encrypted!)
  api_key: c2VjcmV0a2V5              # echo -n 'secretkey' | base64
```

Q: "Are Kubernetes Secrets secure?"

> *"By default, Secrets are base64 encoded — NOT encrypted. They're stored in etcd. To secure them: 1) Enable encryption at rest for etcd, 2) Use RBAC to restrict access, 3) Consider external secret managers (Vault, Azure Key Vault) with operators like External Secrets Operator. Base64 is encoding, not encryption — anyone with cluster access can decode them."*

HPA + ResourceQuota + LimitRange

```
# HPA — Scale pods based on metrics
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: web-app-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web-app
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70       # Scale when avg CPU > 70%
---
# ResourceQuota — Limit namespace resources
apiVersion: v1
kind: ResourceQuota
metadata:
  name: team-quota
  namespace: team-a
spec:
  hard:
    requests.cpu: "10"
    requests.memory: 20Gi
    limits.cpu: "20"
    limits.memory: 40Gi
    pods: "50"
---
# LimitRange — Default limits for pods
apiVersion: v1
kind: LimitRange
metadata:
  name: default-limits
  namespace: team-a
spec:
  limits:
  - default:                         # Default limit if not specified
      cpu: "500m"
      memory: "256Mi"
    defaultRequest:                  # Default request if not specified
      cpu: "100m"
```

```
      memory: "128Mi"
    type: Container
```

NetworkPolicy + ServiceAccount

```
# NetworkPolicy — Restrict pod communication
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: api-network-policy
  namespace: production
spec:
  podSelector:
    matchLabels: { app: api }
  policyTypes: ["Ingress", "Egress"]
  ingress:
  - from:
    - podSelector:
        matchLabels: { app: frontend }   # Only frontend can reach API
    ports:
    - port: 8080
  egress:
  - to:
    - podSelector:
        matchLabels: { app: postgres }   # API can only reach postgres
    ports:
    - port: 5432
---
# ServiceAccount
apiVersion: v1
kind: ServiceAccount
metadata:
  name: app-sa
  namespace: production
automountServiceAccountToken: false   # Don't auto-mount — security
```

InitContainer + Sidecar

```
apiVersion: v1
kind: Pod
metadata:
  name: app-with-init-and-sidecar
spec:
  initContainers:                    # Runs BEFORE main container
  - name: wait-for-db
    image: busybox
    command: ['sh', '-c', 'until nc -z postgres-svc 5432; do sleep 2; done']
    # Waits until postgres is reachable before starting app

  containers:
  - name: app                        # Main container
    image: myapp:v1
    ports: [{ containerPort: 8080 }]

  - name: log-shipper                # Sidecar — runs alongside main
    image: fluent/fluent-bit:2.1
    volumeMounts:
    - name: app-logs
      mountPath: /var/log/app

  volumes:
  - name: app-logs
    emptyDir: {}                     # Shared between app and sidecar
```

Deployment Strategies

```
# Rolling Update (default — zero-downtime)
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxUnavailable: 0               # Keep all old pods until new ready
    maxSurge: 1                     # Add 1 new pod at a time

# Blue-Green (using service selector switch)
# Deploy v2 alongside v1, then switch service selector
# kubectl patch svc web-app -p '{"spec":{"selector":{"version":"v2"}}}'

# Canary (gradual rollout)
# Deploy v2 with fewer replicas, split traffic
# v1: 9 replicas, v2: 1 replica = 10% canary
# Monitor metrics, then scale v2 up, v1 down
```

Kubernetes Troubleshooting — 100 Scenarios

Pod Failures (1-25)

#	Symptom	Commands	Root Cause	Fix
1	CrashLoopBackOff	`kubectl logs <pod> --previous`	App crash, bad config, missing env var	Fix app code/config, check env vars
2	ImagePullBackOff	`kubectl describe pod`	Wrong image name, no registry auth	Fix image name, create imagePullSecret
3	OOMKilled (exit 137)	`kubectl describe pod` → last state	Memory limit too low	Increase memory limit or fix leak
4	Pending (no nodes)	`kubectl describe pod` → Events	Insufficient resources	Scale nodes or reduce requests
5	Pending (node affinity)	`kubectl describe pod`	No node matches nodeSelector/affinity	Fix labels or add matching nodes
6	Pending (taints)	`kubectl describe pod`	Node has taint, pod lacks toleration	Add toleration to pod spec
7	Pending (PVC)	`kubectl get pvc`	PVC not bound	Check StorageClass, provision PV
8	Init:Error	`kubectl logs <pod> -c <init-container>`	Init container failed	Fix init container command
9	CreateContainerConfigError	`kubectl describe pod`	Missing ConfigMap or Secret	Create the missing resource
10	RunContainerError	`kubectl describe pod`	Bad command, missing entrypoint	Fix container command
11	Evicted	`kubectl describe pod`	Node disk/memory pressure	Clean up disk, increase node resources
12	Error (exit code 1)	`kubectl logs <pod>`	Application error	Check application logs
13	Completed (not restarting)	Check restartPolicy	`restartPolicy: Never` set	Use `Always` for long-running services
14	Terminating (stuck)	`kubectl describe pod`	Finalizer stuck, or PV not releasing	`kubectl delete pod --force --grace-period=0`

#	Symptom	Commands	Root Cause	Fix
15	Unknown	`kubectl get nodes`	Node lost contact	Check kubelet on the node
16	Container keeps restarting	`kubectl logs --previous`	Liveness probe failing	Fix probe endpoint or increase thresholds
17	Pod slow to start	Check readiness probe timing	Large image pull or slow startup	Pre-pull images, use startup probe
18	Multi-container: one keeps crashing	`kubectl logs <pod> -c <container>`	Specific container issue	Debug that container only
19	Pod can't resolve DNS	`kubectl exec -- nslookup kubernetes`	CoreDNS issue	Check CoreDNS pods in kube-system
20	Pod can't reach external internet	`kubectl exec -- wget google.com`	NetworkPolicy blocking egress	Check/fix NetworkPolicy
21	Resource limits not applied	`kubectl describe pod`	LimitRange missing	Create LimitRange in namespace
22	Pod scheduled on wrong node	Check nodeSelector/affinity	Wrong labels	Fix labels/selectors
23	Env var is empty	`kubectl exec -- printenv`	ConfigMap/Secret key wrong	Verify key names in ConfigMap/Secret
24	Volume mount permission denied	Check securityContext	Container runs as wrong UID	Add `fsGroup` or `runAsUser`
25	Pod can't write to volume	Check PVC accessMode	ReadOnlyMany but needs ReadWrite	Use ReadWriteOnce or ReadWriteMany

Service & Networking (26-50)

#	Symptom	Commands	Root Cause	Fix
26	Service has no endpoints	`kubectl get endpoints <svc>`	Label selector mismatch	Fix service selector to match pod labels
27	Service not reachable	`kubectl get svc`, `kubectl get ep`	No running pods match selector	Fix pod labels or selector
28	Ingress returns 404	`kubectl describe ingress`	Path/host mismatch	Fix ingress rules
29	Ingress returns 502	Check backend pod health	Backend pod not ready	Fix readiness probe
30	Ingress TLS not working	Check TLS secret	Secret missing or wrong format	Create proper TLS secret
31	DNS resolution slow	Check CoreDNS metrics	CoreDNS overloaded	Scale CoreDNS, add node-local DNS
32	Pods can't communicate across namespaces	Check NetworkPolicy	NetworkPolicy blocking cross-namespace	Allow namespace in policy

#	Symptom	Commands	Root Cause	Fix
33	Service returns connection refused	Verify targetPort	targetPort doesn't match container port	Fix targetPort
34	LoadBalancer stuck at Pending	`kubectl describe svc`	Cloud provider issue	Check cloud quota/permissions
35	External traffic blocked	Check NSG/security groups	Cloud-level firewall	Allow port in cloud firewall
36	Headless service not resolving	Check StatefulSet	Pod DNS format: `pod-0.svc.ns.svc.cluster.local`	Use full DNS name
37	kube-proxy not routing	Check kube-proxy pods	kube-proxy crashed	Restart kube-proxy DaemonSet
38	CNI plugin failure	Check CNI pod logs	Flannel/Calico crash	Restart CNI pods, check node config
39	NodePort not accessible	Check node firewall	Firewall blocking port range	Open 30000-32767
40	ExternalName service not working	Check DNS resolution	Needs CoreDNS CNAME support	Verify ExternalName value
41	Session affinity not working	Check service config	Missing `sessionAffinity: ClientIP`	Add sessionAffinity
42	Slow service response	Check pod resources	CPU throttling	Increase CPU limits
43	Intermittent 503 errors	Check pod readiness	Pod cycling during updates	Fix rolling update strategy
44	gRPC through ingress fails	Check ingress annotations	Nginx needs gRPC annotation	Add `nginx.ingress.kubernetes.io/backend-protocol: GRPC`
45	WebSocket through ingress drops	Check timeout settings	Default timeout too low	Increase proxy-read-timeout
46	IP whitelisting not working	Check ingress annotations	Wrong annotation syntax	`nginx.ingress.kubernetes.io/whitelist-source-range`
47	Rate limiting not applied	Check ingress config	Missing annotation	Add rate-limit annotations
48	CORS issues	Check ingress CORS config	Missing CORS headers	Add CORS annotations to Ingress
49	SSL redirect loop	Check TLS termination	Both ingress and app doing TLS	Terminate at one layer only
50	Multiple ingress controllers conflict	Check ingressClassName	Both controllers claiming same ingress	Specify `ingressClassName` explicitly

Cluster & Node (51-75)

#	Symptom	Root Cause	Fix
51	Node NotReady	kubelet down	`systemctl restart kubelet`, check logs
52	Node disk pressure	Disk full	Clean images: `docker/crictl system prune`
53	Node memory pressure	RAM exhausted	Evict pods, scale node pool
54	Node PID pressure	Too many processes	Investigate runaway containers
55	etcd slow	Disk I/O bottleneck	Move etcd to SSD, monitor latency
56	API server unreachable	Certificate expired	Renew certs: `kubeadm certs renew all`
57	Scheduler not scheduling	Scheduler pod crashed	Check `/etc/kubernetes/manifests/kube-scheduler.yaml`
58	Controller manager down	Config error	Check static pod manifest
59	kubectl connection refused	kubeconfig wrong	Verify cluster endpoint and certs
60	RBAC permission denied	Missing RoleBinding	Create Role + RoleBinding
61	Can't create resources	ResourceQuota exceeded	Check quota: `kubectl get quota`
62	HPA not scaling up	Metrics server missing	Install metrics-server
63	HPA not scaling down	Cooldown period	Wait 5 min (default stabilization)
64	PVC stuck at Pending	No matching PV or StorageClass	Create PV or fix StorageClass
65	PV not releasing	`persistentVolumeReclaimPolicy: Retain`	Manually clear and release PV
66	Node cordoned accidentally	`kubectl cordon` was run	`kubectl uncordon <node>`
67	Pods not rescheduled after node drain	PDB blocking	Check PodDisruptionBudget
68	Admission webhook blocking	Webhook rejecting requests	Check webhook logs, bypass with dry-run
69	Namespace stuck at Terminating	Finalizer stuck	Remove finalizer from namespace
70	Secret not updating in pod	Pod doesn't re-read secrets	Restart pod or use volume-mounted secrets (auto-update)
71	ConfigMap change not reflected	Pod cached old value	Restart pod or use volume mount
72	Rolling update stuck	New pods failing readiness	Check new image, rollback: `kubectl rollout undo`
73	Helm release failed	Pre-install hook failed	`helm rollback`, fix hook
74	CRD not found	CRD not installed	`kubectl apply -f crds/`
75	Webhook timeout	Webhook service unreachable	Check webhook service and endpoints

Advanced (76-100)

#	Symptom	Root Cause	Fix
76	Pod security policy blocking	PSP/PSA restrictions	Update pod security context
77	Sidecar not terminating	Main container exits, sidecar runs	Use native sidecar containers (K8s 1.28+)
78	Jobs not completing	Container hangs	Add `activeDeadlineSeconds`
79	CronJob missed schedule	>100 missed schedules	Set `startingDeadlineSeconds`
80	Service mesh (Istio) breaking traffic	mTLS misconfigured	Check PeerAuthentication policy
81	Prometheus can't scrape pods	Missing annotations	Add `prometheus.io/scrape: "true"`
82	Grafana dashboard empty	Wrong data source	Verify Prometheus URL in datasource
83	Loki not receiving logs	Fluent Bit config wrong	Check Fluent Bit output config
84	Alertmanager not sending alerts	SMTP/Slack config wrong	Test with `amtool`
85	Horizontal scaling too aggressive	HPA threshold too low	Increase target utilization
86	Cluster autoscaler not adding nodes	Node pool max reached	Increase max nodes in pool
87	Image pull slow	Large images, no caching	Use smaller images, pre-pull
88	Pod affinity not respected	Soft affinity (preferred)	Use `requiredDuringScheduling` if strict
89	Topology spread not working	Labels mismatch	Check `topologySpreadConstraints`
90	Init container timeout	Dependency not available	Add timeout and retry logic
91	Container can't read mounted secret	File permissions	Set `defaultMode: 0400` in volume
92	Ephemeral storage exhausted	Too many logs in container	Set `ephemeral-storage` limits
93	kubectl slow	Large cluster, many objects	Use `--field-selector` to filter
94	Audit logs not working	API server audit config missing	Configure audit policy
95	Custom metrics HPA not working	Prometheus adapter not configured	Install prometheus-adapter
96	Blue-green rollback needed	New version has bug	Switch service selector back to old version
97	Canary showing errors	New version failing	Scale canary to 0, keep stable version
98	GitOps (ArgoCD) out of sync	Manual changes in cluster	Enable auto-sync or reconcile manually
99	Priority class evicting pods	Wrong PriorityClass assigned	Review and fix PriorityClass values
100	Cluster upgrade fails	Version skew too large	Upgrade incrementally (1.27→1.28→1.29)